WORKSHOP

Bangun AI Project Manager Anda

Asisten agentik dengan CrewAI dan model open-source lokal — praktik langsung, dari agent pertama hingga kantor virtual

Felix Yustian Setiono · Senior AI / Computer Vision Engineer

Sesi praktik MurzEdu · ~2 jam · Python + laptop yang mampu menjalankan Ollama



Felix Yustian Setiono

Senior AI / Computer Vision (CV) Engineer
AI / ML Systems Architect

Semarang, Jawa Tengah, Indonesia
felixyustian@gmail.com
linkedin.com/in/felixsetiono

SAAT INI

Sigmawave AI Singapore — Senior AI/ML & CV Engineer & CV Team Lead
V-TEKI Indonesia — AI Architect & Technical Lead

PROFIL

Lebih dari 10 tahun mengubah AI menjadi sistem bisnis yang berjalan — dari computer vision dan model prediktif hingga otomasi agentik berbasis LLM. Menjembatani rekayasa nyata dengan pasar Indonesia: regulasi, bahasa, dan operasi UMKM.

KEAHLIAN UTAMA

- **LLM & Agentic Systems:** CrewAI, LangChain / LangGraph, MCP, multi-agent
- **Computer Vision & Edge AI:** YOLO, TensorRT, NVIDIA Jetson, DeepStream
- **Machine Learning & MLOps:** Python, PyTorch, forecasting, FastAPI, Docker
- **Document AI:** OCR, ekstraksi KTP / ID, governed LLM extraction

SOROTAN TERPILIH

- Asisten bisnis 5-agent untuk 65 juta UMKM Indonesia (juara OpenClaw Agenthon 2026)
- Predictive maintenance industri — deteksi kerusakan sebelum terjadi (MachineGuard)
- Governed AI untuk keputusan teregulasi OJK — auditable & deployable (PRANATA)
- Otomasi dokumen / KYC — KTP & faktur terbaca otomatis, aman PII

KREDENSIAL & PENGHARGAAN

M.Eng., ITB · Insinyur Profesional (Ir.) · Google Cloud GenAI Leader · Peneliti terpublikasi (Scholar h-index 4) · Anggota IEEE / PII / KORIKA

Yang Akan Kita Bangun Hari Ini

1. Tujuannya — crew agent yang membantu menjalankan proyek Anda
2. Cara kerja agent: LLM + tools + sebuah loop
3. Menyiapkan CrewAI pada model open-source lokal (Ollama)
4. Membangun asisten PM — langkah demi langkah, dengan kode
5. Intermezzo — kantor virtual berisi agent
6. Membuatnya andal, teramati, dan aman

Yang kita bangun: AI PM Anda

ai-pm — python pm_crew.py

```
$ python pm_crew.py --brief brief.md
[Planner]  breaking the brief into tasks...
[Risk]     top 3 risks, each with a mitigation
[Writer]   kickoff status update drafted

=== RESULT: 10 tasks planned ===
```

TASK	OWNER	DAYS
Meet with design lead to discuss onboarding…	PM	2
Define and document the new 3-step signup p…	PM	3
Create wireframes and mockups of the new si…	Designer	4
Collaborate on implementing design changes,…	Engineer 1	5

```
saved to project_state.json
```

APA YANG DILAKUKAN

— Mengubah project brief menjadi tugas, pemilik, dan estimasi

— Menandai risiko dan menyusun status update

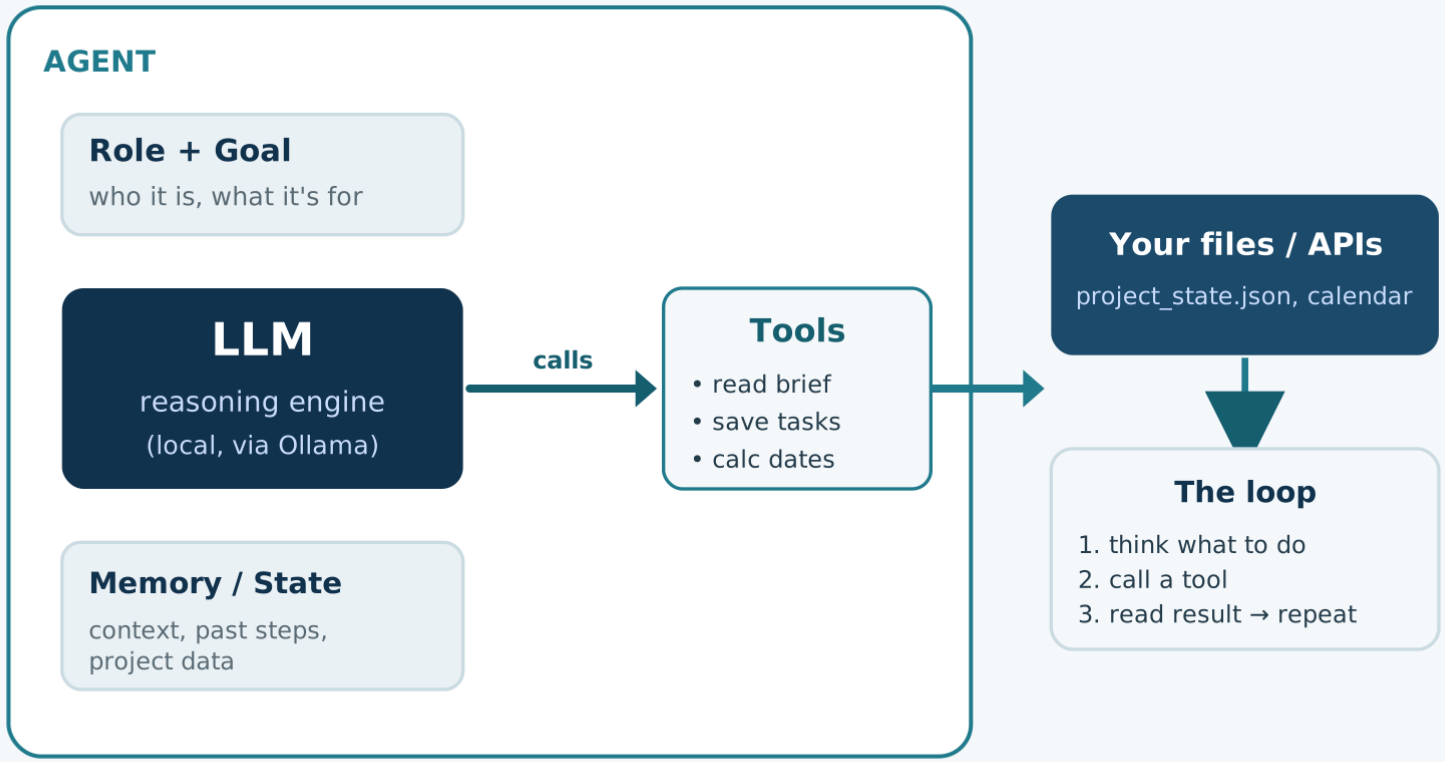
— Berjalan lokal — data Anda tak meninggalkan mesin

Dan Anda membangunnya dengan kode, hari ini.

01 Cara Kerja Agent

LLM + tools + sebuah loop — komponennya

Anatomi sebuah agent



EMPAT BAGIAN

— Sebuah LLM yang bernalar

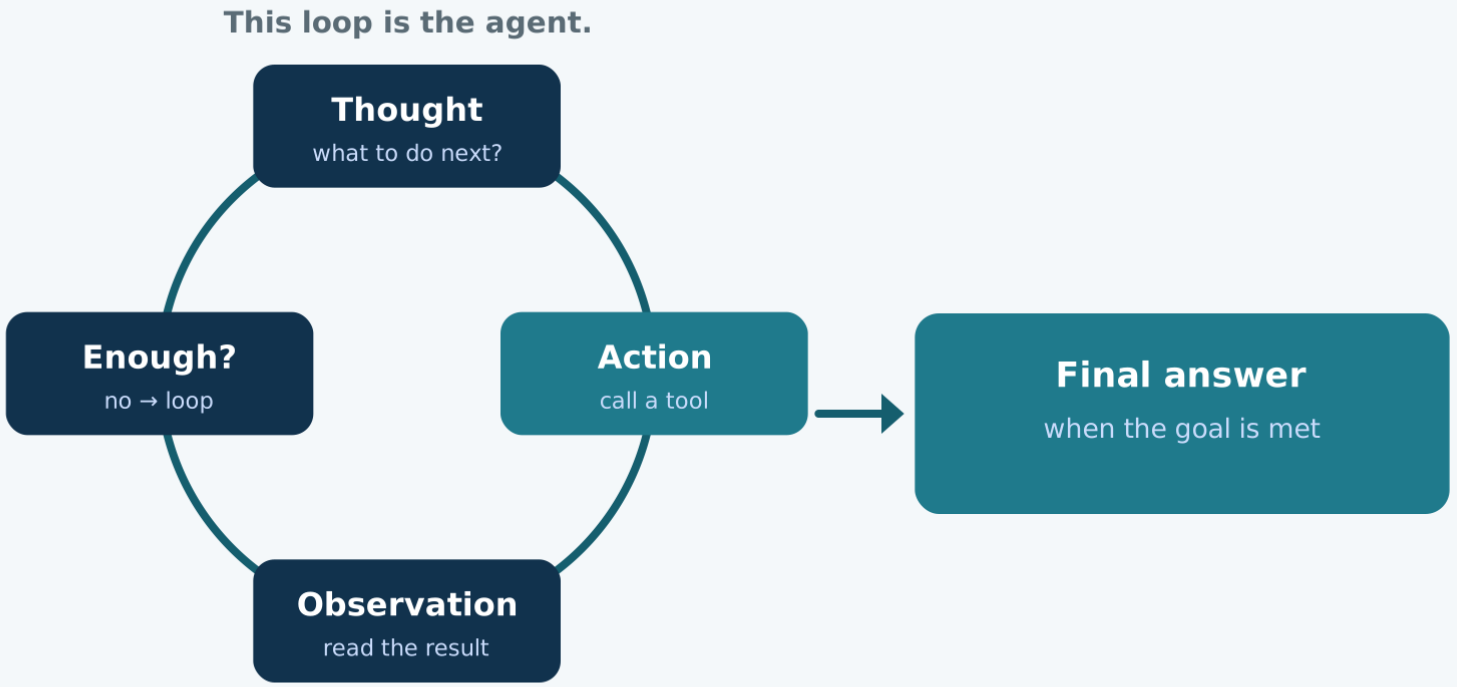
— Tools yang bisa dipanggil (fungsi biasa)

— Memory / state yang dibawanya

— Sebuah loop yang mengikat semuanya

Tanpa hype: agent adalah loop di sekeliling model.

Loop-nya: Reason → Act → Observe (ReAct)



SETIAP PUTARAN

— Memikirkan langkah berikutnya

— Act — memanggil sebuah tool

— Observe hasilnya

— Ulangi hingga tujuan tercapai

CrewAI menjalankan loop ini untuk Anda.

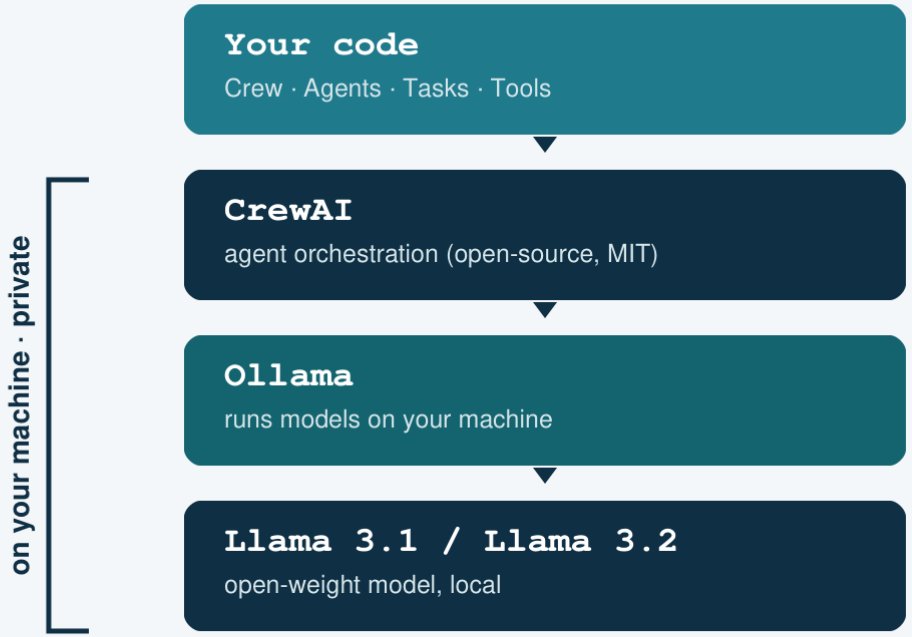
Satu agent, atau satu crew?

- Satu agent cukup untuk satu tugas yang terfokus.
- **Tugas sulit lebih baik dibagi ke spesialis —** planner, researcher, writer.
- Crew mengoordinasikannya: sequential (pipeline) atau hierarchical (manager mendelegasikan).
- Lebih banyak agent = lebih mampu, tetapi lebih mahal, lambat, dan nondeterministik.

PEGANGAN

Mulai dengan satu agent. Tambah peran hanya saat satu agent jelas kewalahan.

Open-source, dan berjalan di mesin Anda



Why local?

- data stays in-house
- no per-token cost
- full control
- works offline

STACK-NYA

— CrewAI mengorkestrasi para agent

— CrewAI terhubung langsung ke Ollama

— Ollama menjalankan model open lokal

— Llama 3.1 / Llama 3.2 — open weights

Tanpa API key. Tanpa data keluar dari laptop Anda.

02

Membangun Asisten PM

Brief masuk; rencana keluar

Setup — Ollama + CrewAI

Instal sekali, tarik model, tambahkan framework-nya. Windows: .venv\Scripts\activate

```
# 1. install Ollama (ollama.com), then pull open models
ollama pull llama3.1          # ~4.7 GB, runs fully local
ollama pull llama3.2:3b       # ~2 GB, for weaker laptops

# 2. a clean Python 3.10-3.13 environment + the framework
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 3. sanity check: is everything ready?
python check_setup.py
```

→ **Dikunci ke crewai==1.15.35 agar sesi dapat direproduksi di berbagai laptop.**

Agent pertama Anda

Agent + Task + Crew, diarahkan ke model lokal.

```
from crewai import Agent, Task, Crew, Process, LLM

# point CrewAI at the LOCAL model served by Ollama
llm = LLM(model="ollama/llama3.1",
          base_url="http://localhost:11434")

planner = Agent(
    role="Project Planner",
    goal="Turn a project brief into a clear task list",
    backstory="A sharp PM who makes vague briefs actionable.",
    llm=llm, verbose=True,
)

task = Task(
    description="Break down this brief into tasks:\n{brief}",
    expected_output="A numbered task list with an owner each",
    agent=planner,
)
```

→ **Satu agent, satu task, satu crew untuk menjalankannya — agent yang berfungsi dalam ~20 baris.**

Beri sebuah tool

Tool hanyalah fungsi Python yang boleh dipanggil agent.

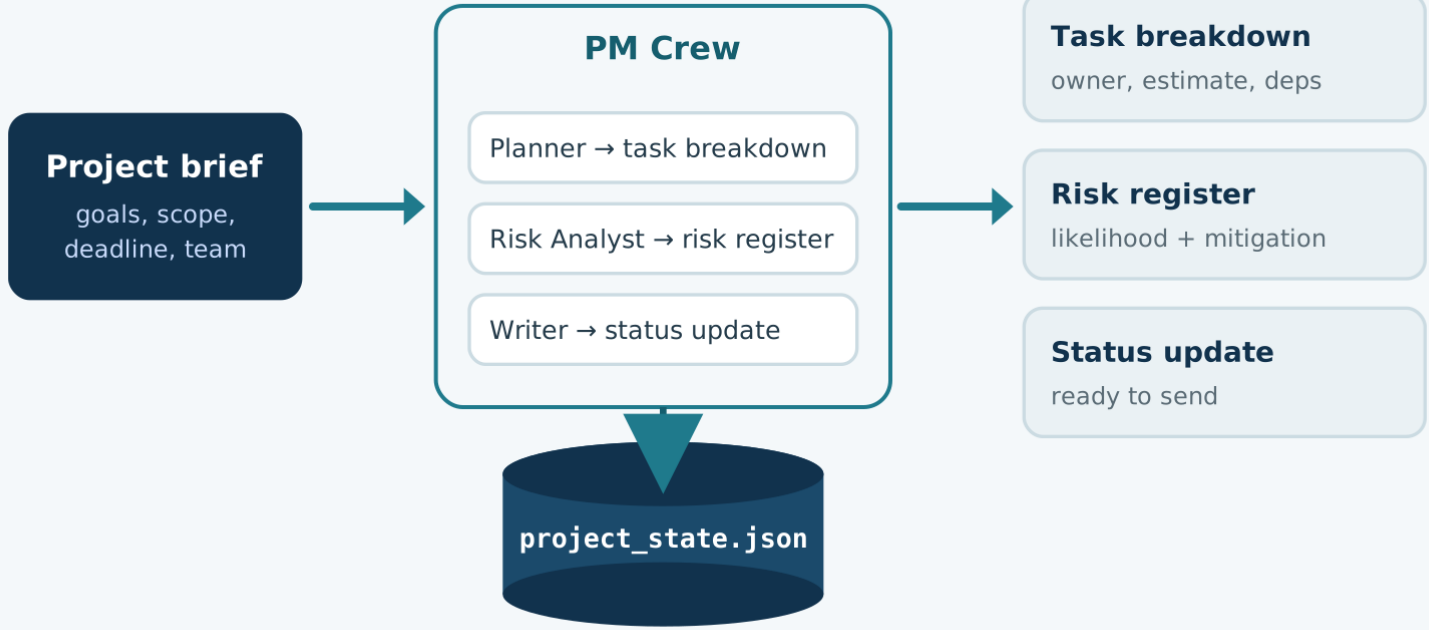
```python
from crewai.tools import tool
import json

@tool("save_tasks", max_usage_count=1)
def save_tasks(tasks_json: str) -> str:
    "Save a JSON list of task titles to tasks.json."
    data = json.loads(tasks_json)
    with open("tasks.json", "w") as f:
        json.dump(data, f, indent=2)
    return f"saved {len(data)} tasks"

# the agent may now decide to call this tool
planner.tools = [save_tasks]
```

→ **Model yang memutuskan apakah memanggilnya. Jalankan tool_demo.py beberapa kali: kadang tidak dipanggil.**

Asisten PM: bentuknya



ALURNYA

- Brief masuk → PM crew
- Planner, Risk Analyst, Writer
- Keluar: tugas, risiko, status update
- State bersama di project_state.json

Tiga spesialis, satu berkas state bersama.

Output bertipe, bukan prosa

Buat planner mengembalikan data terstruktur yang bisa dipercaya langkah berikutnya.

```python
from pydantic import BaseModel
from typing import List

class TaskItem(BaseModel):
    title: str
    owner: str
    estimate_days: int

class Plan(BaseModel):
    tasks: List[TaskItem]

plan_task = Task(
    description="Break the brief into tasks:\n{brief}",
    expected_output="A Plan object",
    agent=planner,
    output_pydantic=Plan,     # validated, typed output
)
```

→ **Skema Pydantic berarti tanpa regex pada prosa — output tervalidasi dan bertipe.**

Tambah dua spesialis

Seorang risk analyst dan seorang status writer, masing-masing satu tugas.

```
risk = Agent(
    role="Risk Analyst",
    goal="Flag the top risks, with one mitigation each",
    backstory="A cautious PM who has watched projects slip.",
    llm=llm)

writer = Agent(
    role="Status Writer",
    goal="Draft a short status update for stakeholders",
    backstory="Writes clear, calm updates and never invents facts.",
    llm=llm)

risk_task = Task(description="Review the plan. List the top risks.",
                 expected_output="A risk register", agent=risk,
                 context=[plan_task])

status_task = Task(description="Write a kickoff update. Work has NOT started: claim no progress.",
                   expected_output="At most 5 lines", agent=writer,
                   context=[plan_task, risk_task])
```

→ **Satu agent = satu tanggung jawab. Jaga peran tetap sempit dan tujuan tajam.**

Jalankan PM crew

Rangkai tiga agent menjadi satu sequential crew lalu jalankan.

```
from pathlib import Path

crew = Crew(
    agents=[planner, risk, writer],
    tasks=[plan_task, risk_task, status_task],
    process=Process.sequential,     # planner -> risk -> writer
    verbose=True,
)

result = crew.kickoff(inputs={"brief": open("brief.md").read()})
print(result)      # the final task's output: the status update

# save shared state from Python, so it can't depend on the model's choice
plan = result.tasks_output[0].pydantic
Path("project_state.json").write_text(plan.model_dump_json(indent=2))
```

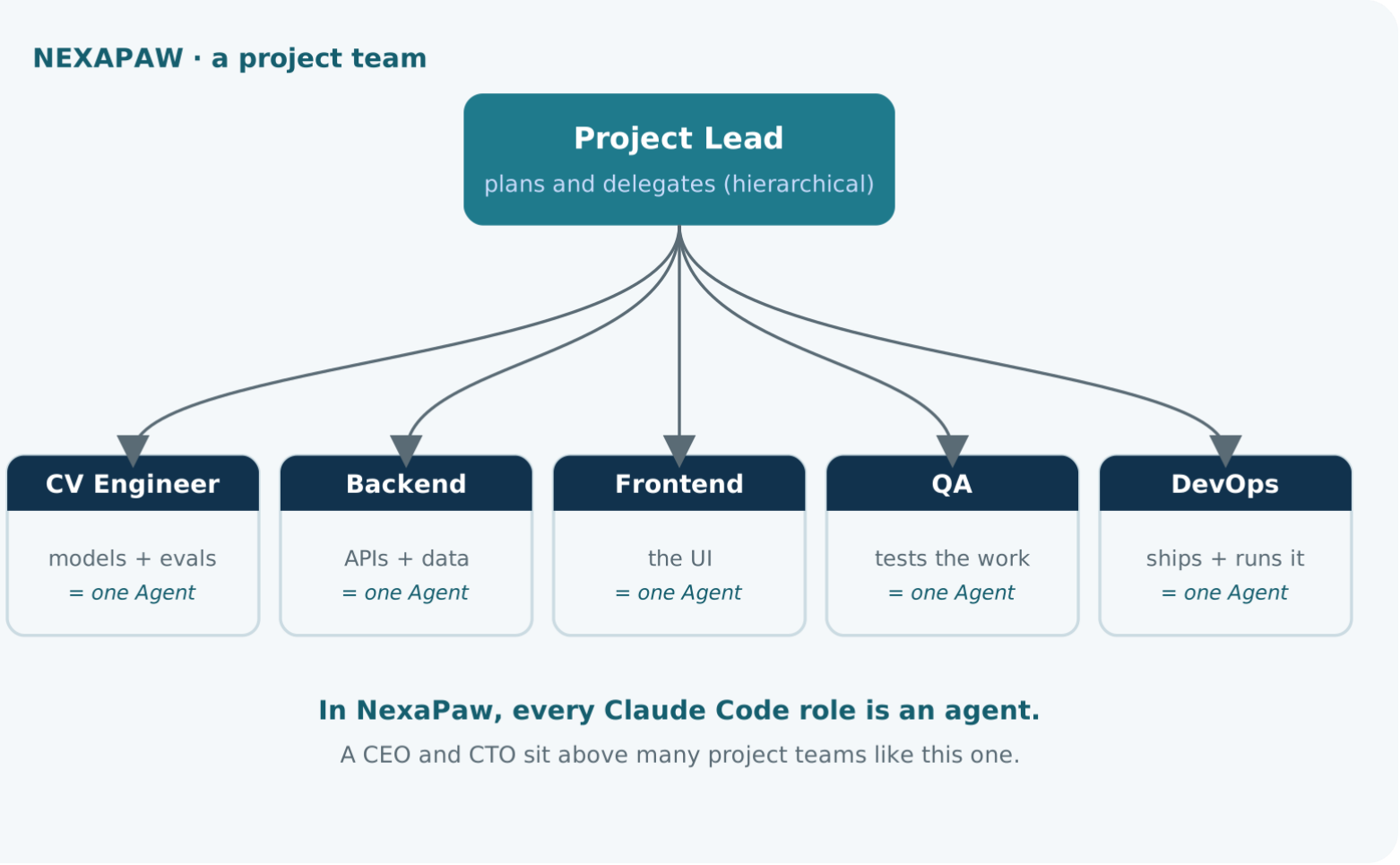
→ **Jalankan pada brief nyata. Itulah AI project manager Anda — sekitar 40 baris.**



Intermezzo · Kantor Virtual

Saat satu asisten menjadi satu tim

NexaPaw: kantor agentik nyata



IDENYA

— Tiap Claude Code role adalah agent — punya tugas, proyek, task

— CEO & CTO di atas para project lead, yang memimpin tim spesialis

— Lead mendelegasikan; tim saling mengoper pekerjaan

— Ini kantor NexaPaw saya — 65 agent, live di HQ 3D

Sebuah perusahaan berisi agent — berjalan di mesin Anda.

Dari crew ke kantor: Lead mendelegasikan

Lead mengembalikan rencana bertipe; Python yang menjalankannya. Anda definisikan staf, bukan langkahnya.

```
class Assignment(BaseModel):      # a role must name a real staff member
    role: Literal["CV Engineer", "Backend Engineer", "Frontend Engineer", "QA Engineer"]
    work: str

class WorkPlan(BaseModel):
    assignments: List[Assignment]

# 1. the Lead decides who does what
planning = Task(description=f"Assign this goal to your team ({', '.join(team)}): {brief}",
                expected_output="A WorkPlan with at most 3 assignments", agent=lead,
                output_pydantic=WorkPlan)
plan = Crew(agents=[lead], tasks=[planning]).kickoff().pydantic

# 2. Python dispatches the Lead's decisions to the specialists
tasks = [Task(description=f"{a.work}. Use at most 4 bullet points.", agent=team[a.role],
              expected_output="At most 4 bullet points") for a in plan.assignments]
result = Crew(agents=list(team.values()), tasks=tasks).kickoff()
print(result)
```

→ **Seperti tim proyek NexaPaw: Lead menentukan siapa mengerjakan apa; peran yang tidak valid gagal divalidasi.**

Yang diberi kantor ini — dan ongkosnya

- Spesialis mengungguli satu generalis pada pekerjaan kompleks berlapis.
- Manager menangani koordinasi yang jika tidak harus Anda hard-code.
- Namun lebih banyak agent = lebih banyak panggilan model = lebih lama dan nondeterministik.
- **Model lokal lebih lambat dari hosted —** sesuaikan ukuran crew dengan hardware Anda.
- Dalam pengujian saya, Process.hierarchical tak pernah sampai ke spesialis pada Llama 3.1 atau 3.2. Periksa siapa yang benar-benar berjalan.

TRADE-OFF JUJUR

Kantor virtual itu kuat dan lebih sulit dikendalikan. Pakai hanya saat satu agent mentok.

03

Membuatnya Nyata

Dari skrip menjadi sesuatu yang bisa diandalkan

Keandalan: guardrail + manusia dalam loop

Validasi output; tahan tindakan berdampak di belakang manusia.

```python
# guardrail: Python validates the output; failures go back to the agent
def short_update(output: TaskOutput) -> tuple[bool, Any]:
    if len(output.raw.split()) > 120:
        return (False, "Too long. Keep it under 120 words.")
    return (True, output.raw)

status_task = Task(description="...", expected_output="...", agent=writer,
                   guardrail=short_update, guardrail_max_retries=2)

# gate risky actions behind a human — draft, never send
@tool("send_email")
def send_email(to: str, body: str) -> str:
    "Save a draft for human approval; does NOT send."
    Path("outbox.md").write_text(body)
    return "DRAFT saved to outbox.md — needs approval"

# stop a stuck agent from looping forever
planner = Agent(role="Project Planner", goal="...", backstory="...",
                llm=llm, max_iter=8, max_retry_limit=2)
```

→ **Agent bisa bertindak. Tahan aksi tak-terbalik — kirim, bayar, hapus — di balik persetujuan manusia.**

Amati, dan kendalikan biaya

- Aktifkan verbose / logging untuk melihat tiap thought, panggilan tool, dan hasil.
- **Lacak token dan latensi —** model lokal gratis tetapi lebih lambat.
- Cache panggilan berulang; jaga prompt dan konteks tetap ringkas.
- Evaluasi pada set brief tetap dengan output yang diketahui benar sebelum dipercaya.

SEBELUM PRODUKSI

Jika tak bisa diukur pada test set, ia tak bisa dipercaya di depan stakeholder.

Agent yang bertanggung jawab & aman

- **Least privilege —** beri agent hanya tool yang benar-benar diperlukan.
- Local-first menjaga data pelanggan dari server pihak ketiga (membantu UU PDP).
- Waspadai prompt injection saat agent membaca teks eksternal, email, atau halaman web.
- **Pertahankan manusia pada setiap keputusan tentang orang —** rekrutmen, kredit, kinerja.

ATURAN SEDERHANA

Agent sebaiknya mengusulkan; manusia yang memutuskan — untuk hal apa pun yang penting.

Seluruh build, dalam tujuh baris

1. Agent adalah LLM plus tools, memory, dan sebuah loop (ReAct).
2. CrewAI menjalankan loop itu; Ollama menjalankan model secara lokal.
3. Agent + Task + Crew adalah asisten berfungsi dalam ~40 baris.
4. Tool hanyalah fungsi Python yang boleh dipanggil agent.
5. Output bertipe (Pydantic) membuat agent bisa dirangkai.
6. Lead + spesialis = kantor virtual (delegasi bertipe).
7. Guardrail, manusia dalam loop, dan eval membuatnya tepercaya.

Tanya jawab & ide build

TANYA JAWAB

Pekerjaan proyek apa yang pertama Anda serahkan ke crew agent?

- **Tool apa yang ia butuhkan —** kalender, repo, tracker?
- Di mana manusia harus tetap dalam loop?

Terima kasih

Tanya Jawab & Ide Build

Felix Yustian Setiono · https://linktr.ee/felixyustian · felixyustian@gmail.com

Bangun AI Project Manager Anda — sesi praktik MurzEdu